TTU Online Admissions Portal

Complete System Documentation

Architecture, Daily Operations, Data Handling,
Identity Matching, and Cybersecurity Measures

Prepared for	April Hollingsworth
Department	TTU Online, eLearning & Academic Partnerships
Platform	Web application (mobile-first, desktop-ready)
Database	PostgreSQL (`lumen_warehouse`)
Deployment	`tosmonline0001.ttu.edu`
Version	1.0 — October 9, 2026

Internal document. Contains operational details — distribute on a need-to-know basis.

Contents

1 What This System Is **4**

1.1 The Big Picture 4

1.2 The Problem It Solves 5

1.3 What It Looks Like 5

2 The People and Their Roles **6**

2.1 Four Levels of Access 6

2.2 How Enforcement Works 6

3 The Student Journey, Step by Step **8**

3.1 Step 1 — Signing Up 8

3.2 Step 2 — The Identity Match 8

3.3 Step 3 — The Application (RaiderConnect) 10

3.4 Step 4 — The Checklist 10

3.5 Step 5 — Documents 11

3.6 Step 6 — Messaging 11

3.7 Step 7 — Sharing the Moment 12

4 The Staff Side: The Admin Portal **13**

4.1 Dashboard 13

4.2 Leads 13

4.3 Applications 13

4.4 Person 360 . . . 13
4.5 Match & Merge . . . 14
4.6 Review Queue . . . 15
4.7 Status Changes and the Outbox . . . 15
4.8 Live Chat . . . 15
4.9 Users & Roles . . . 15
4.10 Audit Trail . . . 15
4.11 Analytics . . . 16

5 Where the Data Lives **17**

5.1 Three Zones, Three Responsibilities . . . 17
5.2 The Key Tables, in Plain Language . . . 18
5.3 How Sync Works — Clone Down, Queue Up . . . 19

6 Cybersecurity, Explained for Non-Engineers **21**

6.1 The Layer Cake . . . 21
6.2 Layer 1 — Passwords . . . 21
6.3 Layer 2 — Multi-Factor Authentication . . . 21
6.4 Layer 3 — Sessions (Staying Logged In) . . . 22
6.5 Layer 4 — Rate Limiting (Brute-Force Armor) . . . 22
6.6 Layer 5 — Forgery Protection (CSRF & CORS) . . . 23
6.7 Layer 6 — Security Headers . . . 23
6.8 Layer 7 — Injection Defense . . . 24
6.9 Layer 8 — Files (Documents) . . . 24
6.10 Layer 9 — Encryption in Transit . . . 25
6.11 Layer 10 — Watching Everything (Audit) . . . 25
6.12 Layer 11 — Boot-Time Safety Checks . . . 25
6.13 The Deliberate Air-Gap . . . 25

7 Running the System Day to Day **26**

7.1 What Staff Do Each Morning 26
7.2 The Normal Data Flow 27
7.3 Capacity 27

8 Production — `tosmonline0001.ttu.edu` 28
8.1 The Shape of Production (as deployed) 28
8.2 Go-Live Checklist (what was actually done) 29
8.3 What Happens If Something Breaks 30

9 Open Items & Known Trade-offs 31
9.1 Still To Receive / Decide 31
9.2 Honest Limitations 31

Glossary 32

PART 1

What This System Is

1.1 The Big Picture

The TTU Online Admissions Portal is a web application that connects three groups of people and one very large database:

- **Prospective students** — people who filled out a "Request for Information" (RFI) form at an event, on the phone, or online. Their details already live in our Salesforce system.
- **Applicants** — students who create an account in this portal and actually apply to a TTU Online program, upload documents, and track their progress.
- **Staff** — advisors who review transcripts and documents, managers who combine duplicate records, and administrators who oversee the entire operation.
- **The Salesforce mirror and its clone** — a copy of our Salesforce data lives in our own PostgreSQL database (`lumen_warehouse`) as the `sf_origin` schema. A sync engine inside the portal re-copies just the TTU Online slice into the app's own `AH.sf_*` clone tables every 5–30 minutes. The portal reads only its clone, and — very deliberately — never writes to the mirror automatically.

In one sentence

The portal gives every applicant a clean, mobile-friendly place to apply and upload documents, while giving staff a back-office control room where every lead, application, document, and decision is visible — and where every change bound for Salesforce is queued for a human to approve by hand.

1.2 The Problem It Solves

Before this portal, the workflow had a painful gap. A person might give us their email and phone number at a recruiting event (becoming a "lead" in Salesforce), then months later apply for a program — possibly with a different name spelling, a new phone number, or a different email address. Salesforce would then hold *two* records that belonged to the *same* human, and staff had to hunt them down manually.

The portal does three things about this:

1. When a student signs up, it *automatically* looks for their existing Salesforce record. A perfect email *and* phone match links instantly. A partial match goes to a staff review queue.
2. Staff get a dedicated "match" screen to confirm or reject those links, and to manually tie a portal account to a Salesforce record that the computer could not find.
3. **No automatic changes to Salesforce.** Every merge, link, or status change lands in a supervised "outbox" that a superadmin reviews and applies one by one.

1.3 What It Looks Like

The interface is deliberately minimal: black, white, and Texas Tech red used only as an accent. Sharp corners, strong typography, and big readable tap targets. It is designed phone-first — most prospective students will open it on a phone — but adapts to a full sidebar layout on desktop monitors.

Two personality touches keep it from feeling like a sterile government form:

- **Raider Red** appears on the student dashboard in three moods: sitting patiently when you haven't applied yet, pointing you forward while you're working through requirements, and dancing under a rain of digital confetti when everything is done.
- **Raider Red is also the chat assistant** — when a student asks a question and no advisor has claimed the conversation yet, a bot persona answers and flags staff to jump in.

PART 2

The People and Their Roles

2.1 Four Levels of Access

Every person who logs in belongs to exactly one role. The role decides which screens exist for them and which actions they can perform.

Role	What they can do
Student	Apply, upload documents, check requirements, message staff, see their own linked university record, update their profile.
Advisor	Everything visible in the admin area that involves reading: leads list, applications list, person profiles, document review (approve / reject / waive), and live chat.
Manager	Everything an advisor can do, *plus* the match & merge screens, status changes (which queue Salesforce work), and analytics.
Administrator	Everything: staff account creation, role changes, disabling accounts, the full audit trail, the Salesforce outbox, and user management.

Why roles matter

An advisor who only reviews transcripts has no business merging identity records or reading the audit log. Role-based access means each employee sees the smallest set of tools needed for their job — which shrinks both mistakes and attack surface if an account is ever compromised.

2.2 How Enforcement Works

Roles are enforced in *two* independent places, not just one:

1. **At the front door.** Every request to an administrative page is checked before any page code

runs at all. A student's browser never even receives the administrative page content — they are simply redirected to their own portal.

2. **At every action.** Every request that reads or changes data checks the caller's role again. Even if someone crafted a hidden link or typed a URL directly, the server answers *"forbidden"* (HTTP 403).

This was tested deliberately: advisors attempting manager-only actions, and students attempting any administrative action, are all blocked.

PART 3

The Student Journey, Step by Step

3.1 Step 1 — Signing Up

A new student visits the portal and creates an account with their email, phone number, name, and a password. Three things happen in that single moment:

1. **The account is created** in our own project tables (a dedicated area of the database called the `AH` schema — the project's private workspace inside the shared warehouse).
2. **The password is transformed** into a one-way cryptographic hash. The real password is never stored anywhere — only a mathematical fingerprint that cannot be turned back into a password.
3. **The identity match runs.** The system quietly compares the email and phone number against nearly fifty thousand sampled Salesforce contacts (and millions in production) to answer one question: *have we heard from this person before?*

3.2 Step 2 — The Identity Match

This is the heart of the system. The email a student signs up with may or may not be the one they gave us as a lead — so at signup the matcher scores the new account against the cloned contact slice on four independent signals: **email** (40 pts, checked against every email field on the SF record), **phone** (35, against every phone field), **last name** (15, including former last names — people change surnames), and **first name** (10, including nicknames and preferred names). The total is a **confidence score** from 0 to 100. (The SF record's mailing address is shown to the reviewer as context but isn't scored — the portal doesn't collect one yet.) Three outcomes are possible:

Match	Result	What the student sees
Email + phone + a name all match	**Auto-linked** (`auto_full`)	Their existing university record appears on the dashboard immediately.
Email + phone match, name doesn't	**Auto-linked + flagged** (`auto_pair`)	Works immediately, but the link sits in a flagged lane of the match queue until a human eyeballs it — a name that disagrees is how duplicates sneak through.
Exactly one field hits	**Pending review** (`auto_partial`)	"We're verifying your record"; a manager picks the right candidate from the scored list (every candidate, not just the top one).
Nothing matches	**Unlinked**	Normal portal; the match queue offers a search that scans *all* warehouse contacts, not just the online slice — a student's lead may predate it.

Example. Maria gave us `maria.g@gmail.com` and phone (806) 555-0117 at a college fair last spring. This fall she signs up with the same email but a new phone number. The system finds the email match, flags the phone mismatch, and places the link in the review queue. A manager looks at both records side by side, calls Maria to confirm, and clicks *Confirm*. Only then is the link made permanent — and a work item appears in the outbox so a superadmin can reconcile the duplicate inside Salesforce itself.

3.3 Step 3 — The Application (RaiderConnect)

The actual application — demographics, disciplinary questions, program and term selection — is completed on **RaiderConnect** (`texastech.my.site.com/online`), Texas Tech's official applicant portal. That is deliberate: Salesforce owns the application record, and RaiderConnect is where it must be created.

RaiderConnect cannot be embedded in our page — Salesforce responds with `X-Frame-Options: SAMEORIGIN` and a `frame-ancestors` CSP that forbids framing — so the portal hands the student over in the same tab and keeps the seam invisible:

1. The student picks **modality** (8-week or 16-week courses — 16-week is Education only) and **program** from the 37-entry catalog mirrored from RaiderConnect's own picker, plus a start term. This saves an *intent* row (`rc_started`) and a timeline event.
2. *Continue to RaiderConnect* takes them to the official form in the same tab, with on-page guidance that mirrors the picks they just made ("choose Undergraduate → Fully Online → 8-week → *their program*").
3. When they come back — detected automatically on tab focus, a 30-second poll, or the *I've submitted — check now* button — the portal looks for a new TTU Online application on their linked contact, live in the warehouse first (fresher than the clone). When it lands, the intent row attaches the Salesforce application id and flips to `submitted`.

3.4 Step 4 — The Checklist

Once the Salesforce application exists, the checklist *is Salesforce's*: every `TargetX_SRMb__Checklist_Item__c` row for that application is piped into the warehouse (`AH.checklist_items`, via the Lumen "SF Checklist Items" connector, incremental) and cloned into `AH.sf_checklist_item` each sync — name, requirement code, status, received/closed/waived flags. What a student uploaded on RaiderConnect shows up received on the next sync, automatically.

The local `portal_checklist_items` template checklist remains only as a fallback while no SF application exists yet — and for **General Business** applicants the portal shows "requirements coming soon," because that program's requirements are still being finalized on the TTU side.

A student can also upload straight to the portal for an open requirement; that upload counts as *submitted with us* until Salesforce parity confirms it (next section).

3.5 Step 5 — Documents

Students upload files (PDF, JPG, PNG, HEIC, DOC, DOCX — up to 15 MB each), optionally attached to a specific checklist item. Every upload is:

- **Verified by content, not by name** — the server inspects the file's actual bytes (its "magic bytes") rather than trusting what the phone claims the file type is.
- **Renamed to a random identifier** on disk, so an uploaded file can never be executed as a script or reached by guessing a URL.
- **Encrypted before it hits the disk** (AES-256-GCM — the same class of encryption used by banks) and decrypted only in memory when an authorized person opens it.
- **Access-checked** — a student can open only their own files; staff roles can open anyone's.

Document status is a parity loop, not a single flag:

`pending_review` → `pushed` → `approved` (`rejected` is the sideways exit)

- **pending_review** — lands here the moment it's uploaded; appears in the staff document queue (`/admin/documents`).
- **pushed** — an advisor downloaded the file and attached it to the student's checklist inside RaiderConnect/Salesforce by hand, then marked it pushed (optionally picking which checklist item it satisfies). An outbox ticket records who did it.
- **approved** — set *only* by the sync engine: once the SF checklist item shows received/closed/waived, parity is real and the portal calls it approved. Nobody marks this by hand.

3.6 Step 6 — Messaging

A persistent chat thread connects the student to staff. When a student writes and nobody has claimed the conversation, Raider Red (the bot persona) answers immediately with a helpful holding

message — or, once the custom knowledge-base bot is connected, a real answer. The moment an advisor replies, the thread is "claimed" and the human takes over. Staff see a shared queue of open threads on their side.

3.7 Step 7 — Sharing the Moment

Once an application exists, a *"Tell everyone"* button appears. On a phone it opens the native share sheet (so the student can post "I just applied to TTU Online!" to Instagram or text it to family); on a computer it copies a link to the clipboard.

PART 4

The Staff Side: The Admin Portal

4.1 Dashboard

The admin landing page is a control-room summary: counts of students, applications, leads, pending reviews, unread chat, and a live feed of the most recent events across the whole system.

4.2 Leads

The leads list is a searchable, filterable window onto every RFI contact in the cloned slice — the same records the recruiting team creates in Salesforce, at most one sync interval old. Staff search by name, email, or phone, and filter by lifecycle stage (Prospect, Inquiry, Applicant…) and term.

4.3 Applications

Every application known to the system — both the ones students filed through this portal and the ones already sitting in Salesforce — with status, stage, term, program, and record type. Clicking any row opens the person view.

4.4 Person 360

One page per human. It combines:

- Salesforce contact fields (name, email, phone, status, term),
- every application on file and its stage,
- every recorded interaction (RFI events, touchpoints),

- the linked portal account, if one exists,
- the portal application's **status timeline** — every transition with actor and timestamp, from draft through decision,
- uploaded documents and checklist state,
- staff notes,
- and that person's own activity on the website — which pages they visited and how much time they spent.

This is where an advisor can answer, in seconds, "who is this person and where are they in the process?"

4.5 Match & Merge

The manager-facing screen has three lanes:

1. **Pending review** — partial matches awaiting a human yes/no, with both records shown side by side and exactly which fields matched highlighted.
2. **Unlinked users** — portal accounts with no Salesforce match, each with the system's best-guess candidate contacts attached.
3. **Auto-linked** — the perfect matches that linked themselves, so staff can spot-check that automation behaved.

Every suggested pair carries a **confidence score** (0–100) and per-field evidence chips — email, phone, last name (*including former last names, which catches post-marriage surname changes*), and first name — while the contact's city/state sits alongside as extra context, so the reviewer sees *why* the system suspects a match, not just that it does. A free-text search over the clone covers the case where the algorithm missed entirely.

Important

Confirming a link or requesting a merge does **not** modify Salesforce. It writes a work item into the outbox. A superadmin applies each change by hand, one at a time, inside Salesforce proper.

4.6 Review Queue

A single inbox of everything waiting on a human: submitted checklist items and freshly uploaded documents. Approve, reject (with a note the student sees), or waive — every click is recorded with the advisor's name.

4.7 Status Changes and the Outbox

When a manager changes an application's status — say, from *submitted* to *under review* or *admitted* — three writes happen:

1. the portal's own copy is updated so the student sees progress instantly;
2. a row is appended to `portal_application_events` — the append-only timeline recording *from-status, to-status, actor, timestamp*, so an application's full progression is never lost;
3. a row is appended to the **outbox** describing exactly what must be mirrored into Salesforce.

The outbox is a screen of pending work items for the superadmin: entity, action, payload, who requested it, and when. Each item can be marked *done* or *skipped* with a note. This is the deliberate air-gap between the public website and the institutional system of record: **the website cannot change Salesforce; only a human can.**

4.8 Live Chat

The staff console lists every open student conversation, shows the last message and whether Raider Red is still holding the thread, and lets any advisor claim, reply, or close. Replying automatically claims the thread.

4.9 Users & Roles

Administrators create staff accounts, change roles, and disable accounts. A disabled account is locked out immediately at every layer.

4.10 Audit Trail

Every meaningful thing that happens is written to an append-only activity table with the actor, the action, the detail, the IP address, the browser, and the timestamp:

- every page view and click by *both* students and employees,
- every login, logout, failed login, MFA failure, and password reset (flagged as security events),
- every admin action — reviews, merges, status changes, chat replies,
- and time-on-site per user, powering the analytics screen.

The audit view is admin-only and exists for exactly the questions "who did this?" and "what was this employee doing at 2 pm yesterday?"

4.11 Analytics

A daily event chart, the most-visited pages, and per-user time-on-site — the "how much time did a student spend on the site" requirement made visible.

PART 5

Where the Data Lives

5.1 Three Zones, Three Responsibilities

Everything sits in one PostgreSQL database called `lumen_warehouse`, divided into clearly separated areas ("schemas"):

Zone	Contents
`sf_origin`	The Salesforce mirror: contacts (3.5M), applications (470K), interactions (4.3M), accounts, terms, programs, record types. Fed by the warehouse's own ETL. The portal **reads** it during syncs and **never writes** to it.
`AH.sf_*`	The project's **clone** of the TTU Online slice — contact, application, term, program, record type — refreshed every 5–30 minutes, plus append-only `*_hist` tables keeping every observed version of every row. All portal reads hit this copy.
`AH.portal_*`	The app's own data: users, sessions, identity links, applications, checklists, documents, chat, notes, audit log, outbox, projections. The only zone the portal writes interactively.

This separation is the project's most important architectural rule. The portal can never corrupt institutional data — its writes are physically confined to its own tables, and in production the database account is granted *read-only* permission on `sf_origin` as a second, belt-and-suspenders layer. Keeping our own clone (rather than querying the mirror live) means the admissions site keeps working even if the warehouse ETL is down, lets us attach portal-specific columns to records, and gives us a searchable append-only history of how each record evolved.

5.2 The Key Tables, in Plain Language

Table (in AH)	What it holds
portal_users	Accounts: email, hashed password, role, MFA flag, failed-login counter, lock timestamp.
portal_sessions	Active sign-ins: who, when, from which IP/device, expiry.
portal_otp	Text-message codes: stored as hashes, 10-minute life, 5-try limit.
portal_identity_links	The bridge: which portal account belongs to which Salesforce contact, how it matched, who reviewed it.
portal_applications	Applications filed in the portal (program, term, status, answers).
portal_checklist_items	Each applicant's requirement list and its state.
portal_checklist_templates	The editable requirement lists per program/term — where the real admissions requirements will live.
portal_documents	Document metadata (the files themselves live, encrypted, on disk).
portal_chat_threads/messages	Conversations with advisors and the bot.
portal_outbox	The queue of changes a superadmin must apply to Salesforce by hand.
portal_activity	The audit trail: every click, page, login, and admin action.
portal_notes	Staff notes attached to people.
portal_application_events	Append-only status timeline per application — every transition with actor and timestamp (stateful, not just the latest status).
portal_contact_projection	The "generated columns": for each linked SF contact, the portal user id, latest portal application + status, the full status-history JSON, checklist progress, and last portal activity. Rebuilt each sync and on every link/status change.
sf_contact / sf_application	The cloned slice: current state of the records the portal cares about (denormalized, resolved names).
sf_contact_hist / sf_application_hist	Append-only history: every version of every cloned row, keyed by content hash — how an SF record progressed through statuses over time.
sf_term / sf_program / sf_recordtype	Cloned reference data for the pickers and filters.
sf_sync_runs	Every sync logged: trigger (schedule/manual/boot), row counts, duration,

5.3 How Sync Works — Clone Down, Queue Up

The user asked for one rule above all: *do not let the app mutate the warehouse on its own.* Sync therefore runs in two opposite, asymmetric directions:

Down (Salesforce → portal): automatic, on a timer.

1. A sync engine inside the app wakes every **5–30 minutes** (`SYNC_INTERVAL_MINUTES`, default 15) and once at boot if the clone is empty or stale.
2. Each run takes a Postgres advisory lock first, so a scheduled run and a manual *Refresh now* can never overlap.
3. It re-reads just the TTU Online slice of the 3.6-million-row mirror — contacts flagged Distance Learning, `TTU_Online_Student`, online lead sources, TUO campus, or owning a TTU Online application (≈76,500 contacts) — plus their applications (≈33K), per-item checklist rows (≈55K, TTU Online applications only), and the small reference tables — and **upserts by Salesforce Id** into the `AH.sf_*` clone.
4. Every row carries a content hash; when a row differs from last time it is *also* appended to `sf_*_hist` — so the clone is current-state **and** the history is append-only (no state is ever lost, on either side).
5. After the copy, two reconciliation steps run: *detect* attaches a newly-found Salesforce application to the student's intent row (and writes a `submitted` timeline event), and *parity* flips pushed documents to `approved` once their SF checklist item shows received. Then `portal_contact_projection` is rebuilt: the generated columns stitching each linked contact to its portal user, application status + timeline, SF-side status/stage/decision, checklist counts, open items, and pending uploads.
6. Every run lands in `sf_sync_runs`. Managers and admins see run history and a **Refresh now** button at `/admin/sync`.

Up (portal → Salesforce): manual, always.

1. **Writes stay local.** Anything that changes state lands in `AH.portal_*` tables instantly (the portal is always truthful about the portal) — including a `portal_application_events` row for every status transition.
2. **Salesforce changes queue as tickets.** Status flips, merges, and link confirmations each write an outbox row: *entity, action, payload, requester, timestamp.* The superadmin works the queue one item at a time and marks each done; the next scheduled sync then reflects the change back into the clone — closing the loop without the website ever touching Salesforce.

Why a queue instead of a direct sync

Because identity is messy. If the app merged two Salesforce records automatically and got it wrong, un-merging inside Salesforce is a nightmare of re-parenting related objects. A human approving each merge costs minutes and prevents disasters — the right trade at 200–300 daily users.

PART 6

Cybersecurity, Explained for Non-Engineers

6.1 The Layer Cake

Security here is not one lock but a stack of independent defenses — if any one layer fails, the next still holds. What follows is every layer, in plain terms.

6.2 Layer 1 — Passwords

- Passwords are hashed with **scrypt** at deliberately expensive settings (roughly 128 MB of memory work per attempt). Guessing a password by brute force takes a real fraction of a second *per guess* — fast enough for one login, impossibly slow for billions of guesses.
- A **server-side pepper** (a secret string mixed in before hashing, held only in server configuration — never in the database) means that even an attacker who steals the entire user table still cannot test guesses without also stealing the server environment.
- Password rules: at least 10 characters, must include a letter and a number, cannot be a common password, cannot contain the user's own email.
- Old password hashes upgrade silently — when a user with an older hash format logs in successfully, their password is re-hashed at the new standard automatically.
- Login is **timing-safe**: the same amount of hashing work happens whether or not the email exists, so an attacker cannot probe "is this email registered?" by measuring response time.

6.3 Layer 2 — Multi-Factor Authentication

- MFA uses **Twilio Verify** — codes are generated and checked by Twilio, the industry-standard service, not by us.

- Codes expire in **10 minutes** and allow at most **5 attempts** per challenge.
- Only a *hash* of each code is ever stored in our database.
- The portal records every failed MFA attempt as a security event in the audit log.
- In development (no Twilio keys configured) a fixed code `424242` is used — production prints a loud startup warning if Twilio keys are ever missing.

6.4 Layer 3 — Sessions (Staying Logged In)

- A login creates a signed token (like a tamper-proof wristband) stored in a cookie that JavaScript cannot read (`httpOnly`), that only travels on encrypted connections in production (`secure`), and that is never sent to other websites (`sameSite`).
- Every session also has a row in the database with its expiry, IP, and device — so sessions can be revoked instantly.
- Changing a password **kills every existing session** for that account. A stolen laptop stops being logged in the moment the password is reset.
- Sessions expire in 7 days regardless.

6.5 Layer 4 — Rate Limiting (Brute-Force Armor)

Every sensitive doorway has a speed limit. Exceed it and the server answers "slow down" (HTTP 429) with a retry timer:

Action	**Limit**
Login attempts	10 per 5 minutes (per IP), *plus* 5 failures ⇒ 15-min account lock
Account creation	5 per hour (per IP)
Password reset requests	5 per hour (per IP) + 3 per hour per email
MFA code checks	10 per 5 minutes
Document uploads	20 per 10 minutes
Chat messages	60 per minute
Activity tracking pings	120 per minute
Everything else	240 per minute

What this prevents. An attacker who wants to try ten thousand passwords against one account hits the lock at *five*. Someone trying to create thousands of fake accounts stops at five per hour per IP. It converts unlimited-force attacks into mere annoyances.

6.6 Layer 5 — Forgery Protection (CSRF & CORS)

- Every request that *changes* something must arrive with an `Origin` header matching our own site. A malicious page on another domain cannot silently make your browser submit forms here — its requests are refused outright. (Verified: requests forged from a foreign origin get a 403.)
- No "CORS" permissions are granted at all: other websites' JavaScript simply cannot read our API responses.
- Cross-origin isolation headers (`Cross-Origin-Opener-Policy`, `Cross-Origin-Resource-Policy`) prevent other tabs from sharing a window or loading our authenticated resources.

6.7 Layer 6 — Security Headers

Every response carries a set of instructions that harden the browser itself:

Header	In plain terms
Content-Security-Policy	"Only run scripts/styles/images from this site — nothing external is allowed to execute." Blunts injected-script attacks.
Strict-Transport-Security	"Always use HTTPS with this site for the next two years — never fall back to plain HTTP."
X-Frame-Options / frame-ancestors	"Never let this site be embedded in a frame" — defeats clickjacking overlays.
X-Content-Type-Options	"Don't guess file types" — a file we call an image can never be reinterpreted as code.
Referrer-Policy	Pages don't leak full URLs to third parties.
Permissions-Policy	Camera, microphone, and location are disabled — the app never asks for them.
Cache-Control: no-store	Authenticated pages and API answers are never saved in browser or proxy caches.
X-Robots-Tag: noindex	Search engines are told not to index portal/admin/API pages.

The framework also hides its own name (`poweredByHeader` is off) — there's no reason to advertise the exact software stack to scanners.

6.8 Layer 7 — Injection Defense

- **SQL injection is structurally impossible** in the current code: every query sends user data to the database *separately* from the command, as parameters — the database never interprets a name or email as a command.
- **Cross-site scripting (XSS)** is handled by the framework's escaping plus the strict CSP above; user text is rendered as text, never as HTML.
- **Every input is capped**: emails, names, notes, and chat bodies are trimmed to sane lengths so nobody can flood a field with a megabyte of junk.
- **Path traversal is double-checked**: document downloads validate that the requested ID is a well-formed UUID, that the stored filename is one we generated (not something containing "`../`"), and that the resolved file path really is inside the upload directory.

6.9 Layer 8 — Files (Documents)

The riskiest feature of any admissions portal is file upload. The defenses:

1. **Extension allowlist** — only PDF, JPG, PNG, HEIC, DOC, DOCX.
2. **Magic-byte inspection** — the first bytes of the file must actually be a real PDF/image/document signature. A script renamed `transcript.pdf` is rejected.
3. **Random storage names** — the file lands on disk under a UUID filename in a folder outside the public web directory. It can never be addressed by URL or executed.
4. **Encryption at rest** — AES-256-GCM before writing; the server decrypts only in memory when an authorized viewer opens it.
5. **Ownership checks on every download** — a student can open only their own files; staff roles can open all. Verified: a second student attempting to open someone else's document gets 403.
6. **Size cap** — 15 MB, enforced server-side.
7. **Rate limit** — 20 uploads per 10 minutes.

6.10 Layer 9 — Encryption in Transit

- Between browser and server: HTTPS only. The session cookie refuses to travel unencrypted, and HSTS pins the site to TLS for two years.
- Between application and database: TLS is supported via standard PostgreSQL SSL modes (`sslmode=verify-full`) when the two live on separate hosts; on one host, connections never leave the machine.

6.11 Layer 10 — Watching Everything (Audit)

Every login, failed login, MFA failure, password reset, page view, click, upload, review decision, merge, and status change lands in the audit table with who/what/where/when. Security-sensitive events are tagged distinctly so the audit screen can filter for them. This is both a compliance control and a detection tool — unusual patterns (many failed logins, an employee suddenly bulk-viewing records) are visible in one place.

6.12 Layer 11 — Boot-Time Safety Checks

At startup in production, the server *refuses to boot* if the session secret is missing or weak, and prints loud warnings if encryption keys, the password pepper, or Twilio credentials are absent — so the app can never silently drift into an insecure configuration.

6.13 The Deliberate Air-Gap

The deepest security property isn't cryptographic at all: **the app has no code path that writes to the Salesforce mirror**. It's not that writes are restricted — they don't exist. Even a fully compromised application process cannot alter institutional records; at worst it could queue outbox tickets a human would review.

PART 7

Running the System Day to Day

7.1 What Staff Do Each Morning

1. **Advisors** open the document queue (`/admin/documents`), download each `pending_review` file, attach it to the right checklist item inside RaiderConnect, and mark it `pushed` (optionally naming which SF item it satisfies) — and watch the chat console for new conversations.
2. **Managers** check the match queue: confirm or reject pending identity links, manually link the stragglers, and move applications through their statuses — knowing every flip also writes an outbox ticket and lands on the student's status timeline.
3. **Administrators** review the audit trail for anomalies, manage staff accounts, glance at `/admin/sync` to confirm the clone is fresh (or force a refresh), and work the outbox: applying each queued change in Salesforce by hand and marking it done.

7.2 The Normal Data Flow

End-to-end example — RFI to admit:

1. Recruiter collects an RFI at an event → Salesforce lead appears in `sf_origin` via the warehouse ETL.
2. Next sync (or a manager's *Refresh now*) clones the lead into `AH.sf_contact`; any change since last run is appended to `sf_contact_hist`.
3. Student signs up on their phone → the matcher scores them against the clone (email/phone/last/first name) → auto-link, or queued for review with a confidence score.
4. Student picks program + term and continues to RaiderConnect → intent row saved (`rc_started`); the official application is created there, in Salesforce.
5. The next sync (or the return check) finds the new TTU Online application on their contact → the intent row attaches it, flips to `submitted`, timeline event recorded.
6. Student uploads a transcript in our portal → file encrypted, status `pending_review`, advisor sees it in the document queue.
7. Advisor attaches it to the checklist item inside RaiderConnect and marks the portal copy `pushed` → outbox ticket queued.
8. Next sync sees the SF checklist item received → the portal document flips to `approved` by itself. The record shows identical state on both sides, and a human certified every write to Salesforce.

7.3 Capacity

The workload assumptions baked into the design (200–300 users/day, 2,000–5,000 per month) are modest by web standards. The database connection pool (8 connections), in-memory rate limiter, and polling chat are sized for exactly this scale — nothing in the design bottlenecks until orders of magnitude more traffic. The one component that *will* want upgrading first at scale is the chat's 2.5-second polling, which can become server-sent events or websockets later without changing the user experience.

PART 8

Production — tosmonline0001.ttu.edu

8.1 The Shape of Production (as deployed)

- **Two boxes, one public face.** The portal app runs on `tosmonline0001` — the public-facing host, where nginx already terminates :443 with the real TTU certificate. The databases stay on `tosmonline0003`: `lumen_warehouse` (SF mirror + `AH.checklist_items` via the *SF Checklist Items* Lumen connector) and `AHPortalDB` (every `portal_*` table plus the `sf_*` clone, schema `"AH"`).
- **Cross-box database access** rides a persistent SSH tunnel — Postgres on 0003 binds localhost only, so PM2 process `ahportal-db-tunnel` keeps `127.0.0.1:55432` → `0003:5432` open (key auth between the boxes already existed). The app's `DATABASE_URL` and `WAREHOUSE_URL` both point at `127.0.0.1:55432`.
- **Two PM2 processes** on 0001 (`pm2 saved`): the tunnel and `april-web` (`next start -p 3210`, `NEXT_BASE_PATH=/sim/AHPortal`).
- **Public URL** `https://tosmonline0001.ttu.edu/sim/AHPortal/`: an nginx `location` block in `chatbot.conf` proxies to `127.0.0.1:3210` and forwards `X-Forwarded-Host` / `-Proto` / `-For`, which the app's CSRF check and logs consume.
- **Least privilege:** the portal's `ahportal` role owns `AHPortalDB.AH` and holds *SELECT-only* grants on `lumen_warehouse.sf_origin` and `AH`. Writes to the mirror fail at the permission level.
- **Fresh server-side secrets** live in `.env.local` on 0001; uploads sit in `~/ahportal/uploads`, outside any web path.

Important

The box's glibc (2.28) predates what Next.js's native SWC binary needs — production builds must run `next build -webpack` (the npm `build` script does this). Public assets must be referenced through `asset()` (adds the `/sim/AHPortal` prefix); bare root-relative `/logo.png`-style paths 404 behind the tenant mount.

Important

Before launch: the dev values currently in `.env.local` (passwords, pepper, encryption key) must be regenerated on the server, and the dev MFA code `424242` disappears automatically once Twilio keys are set.

8.2 Go-Live Checklist (what was actually done)

1. **Provision:** created `AHPortalDB` + least-privilege `ahportal` role — owns the portal schema, `SELECT`-only on `lumen_warehouse.sf_origin` and `AH`.
2. **Secrets:** generated server-side into `~/ahportal/app/.env.local`; staff accounts (`admin@`/`advisor@`/`manager@`) created with generated passwords.
3. **Checklist pipe:** added the Lumen connector *SF Checklist Items → AH* (id `3e0ca7b6`, incremental on `SystemModstamp`) so `TargetX_SRMb__Checklist_Item__c` lands in `AH.checklist_items` — 1.46M rows, all-applications.
4. **Build & run:** `npm ci && npm run build` (webpack), PM2 `april-web` on :3210 plus the `ahportal-db-tunnel` SSH forward to 0003; `pm2 save`.
5. **Mount:** nginx `location /sim/AHPortal/` in `chatbot.conf` → `127.0.0.1:3210` with forwarded-host headers (0001 already owns the public :443 cert).
6. **Verified live:** portal 200 on the real cert, HTTPS login → session → admin stats over clone data, signup POST passes CSRF through the proxy, assets load under the prefix, boot sync confirmed idempotent through the tunnel (0 changes on re-run).
7. **Before real students:** set `TWILIO_*` for live SMS MFA (dev code `424242` disappears automatically once set) and point `BOT_ENDPOINT` at the knowledge-base bot.

8.3 Staff Accounts & Where They Live

Sign in at `https://tosmonline0001.ttu.edu/sim/AHPortal/login`.

Account	Password	Role
admin@ttu.edu	zOKsKDcKBRYBe3	superadmin
advisor@ttu.edu	CTFR3z1MViZGwL	advisor
manager@ttu.edu	MwugOcgLjuVOF	manager

While `TWILIO_*` is unset, MFA challenges accept the dev code `424242`. The same credentials live server-side at `~/ahportal/STAFF_LOGINS.txt` on tosmonline0001 — change them at first login and rotate the file copy if the document is shared.

8.4 What Happens If Something Breaks

Scenario	Result
Database unreachable	Pages error cleanly; no data is fabricated; the pool reconnects automatically.
Twilio outage	OTP sends fail; the user sees an error; sessions already active are unaffected.
Upload disk fills	New uploads refuse with an error; nothing else degrades.
Rate-limit false positive	The legit user waits out the `Retry-After` window (minutes), not a ban.
Forgotten admin password	Another admin can reset it; a superadmin can update the user row directly.
Server reboot	Sessions survive (they live in the database); the app restarts via systemd.

PART 9

Open Items & Known Trade-offs

9.1 Still To Receive / Decide

- **General Business requirements** — the one program whose SF checklist isn't finalized; its applicants see "requirements coming soon" until TTU publishes them.
- **Twilio production keys** for live SMS MFA.
- **The custom chatbot endpoint** (`BOT_ENDPOINT`) — wire-in point is ready: the portal posts the message and user context, expects a reply string.
- **Database role details** for production (which account owns `AH`, what to grant).
- **Lead definition** — currently "all prospective-student contacts"; if the business definition of a lead is narrower, the filter is a one-line change.

9.2 Honest Limitations

- **Chat polls** every 2.5 seconds rather than pushing instantly — invisible to users at this scale, but worth upgrading to websockets if the staff queue grows busy.
- **Rate limits live in server memory** — correct for one machine; if the app ever runs on two machines simultaneously they would need a shared store (the database table is a drop-in).
- **JWT role staleness** — a staff member whose role is reduced keeps their old role until their next login (max 7 days). Immediate revocation happens by disabling the account or deleting their session row.
- **Full-disk encryption** of the database volume is an OS-level server task, not something the application can do for itself.

Glossary

RFI
Request For Information — the first contact a prospect makes (event signup, call, web form), which becomes a Salesforce lead.

RaiderConnect
TTU's official applicant portal (`texastech.my.site.com/online`) — where the application itself is completed and where advisors push student documents into Salesforce.

Parity
The moment the warehouse shows Salesforce reflecting something the portal recorded (e.g. a pushed document whose checklist item is now received) — only then does the portal treat it as final.

Lead
A prospective student in Salesforce who has not yet applied.

Salesforce mirror
A copy of our Salesforce data synchronized into our own database (`sf_origin`) by the warehouse ETL — read-only to us.

Clone (`AH.sf_*`)
The portal's own copy of just the TTU Online slice of the mirror, refreshed every 5–30 minutes, with append-only history tables keeping every version of every row.

Sync engine
The in-app job that refreshes the clone on a timer or on demand from `/admin/sync`; advisory-locked so runs never overlap.

Confidence score
The 0–100 match strength from comparing email, phone, last name (including former names), and first name (including nicknames).

Projection
The generated columns stitched onto a linked SF contact: portal user, application status, status

timeline, checklist progress.

Schema

A named compartment inside a database that keeps tables organized — `sf_origin` holds Salesforce data, `AH` holds ours.

Identity link

The recorded connection between a portal account and a Salesforce contact.

Outbox

The queue of pending changes a superadmin applies to Salesforce manually — the bridge between the app and the system of record.

Hash

A one-way fingerprint of a password — can verify a guess, can never reveal the original.

Pepper

A secret mixed into every password hash, stored only in server configuration.

MFA/OTP

Multi-Factor Authentication / One-Time Passcode — the texted code that proves you also control your phone.

Session

The signed cookie that keeps you logged in between page loads.

Rate limit

A speed limit on requests that stops brute-force attacks.

CSRF

An attack where another website tricks your browser into submitting actions on our site — blocked via origin checks.

CSP

Content-Security-Policy — a whitelist telling the browser exactly which scripts and resources may run.

HSTS

The instruction that tells browsers "HTTPS only, forever."

AES-256-GCM

The encryption algorithm protecting uploaded documents at rest — the same standard used for classified traffic.

Magic bytes

The identifying bytes at the start of a real file (a true PDF always begins `%PDF`) — used to detect faked uploads.

Least privilege

Every account gets only the access its job requires.

RBAC

Role-Based Access Control — permissions follow the role, not the person.